

Self-Learning AI Engineer - Comprehensive Guide

Overview

The Quantum Loop Debugger has been enhanced with sophisticated self-learning capabilities that enable the AI Engineer to continuously improve its performance through experience, feedback, and knowledge acquisition. This enhancement maintains zero-cost operation while adding advanced adaptive behavior.

Key Features

1. MCP-Based Memory System (`assistant/memory_system.py`)

- **Experience Capture:** Automatically records task executions, outcomes, and context
- **Adaptive Prompts:** Generates context-aware prompts based on past successful experiences
- **Pattern Recognition:** Identifies success/failure patterns across different task types
- **Contextual Retrieval:** Finds similar past experiences to inform current decisions

2. Feedback Loop Engine (`assistant/feedback_engine.py`)

- **Automatic Evaluation:** Assesses task performance using multiple criteria
- **Learning from Outcomes:** Analyzes successful and failed attempts
- **Performance Optimization:** Identifies improvement opportunities and triggers optimizations
- **Metrics Tracking:** Comprehensive performance analytics and trend analysis

3. Enhanced AI Engineer Core (`assistant/enhanced_core.py`)

- **Adaptive Behavior:** Modifies approach based on learning insights
- **Interactive Learning:** Supports manual configuration and optimization triggers
- **Self-Improvement:** Automatically applies learned optimizations
- **Backward Compatibility:** Extends the original AI Engineer without breaking changes

4. Knowledge System (`assistant/knowledge_system.py`)

- **Document Processing:** Ingests knowledge from PDFs, text files, URLs, and manual input
- **Gap Identification:** Automatically identifies knowledge gaps from failures
- **Improvement Proposals:** Generates actionable improvement recommendations
- **Knowledge Graph:** Builds relationships between knowledge items

5. MCP Learning Server (`assistant/mcp_learning_server.py`)

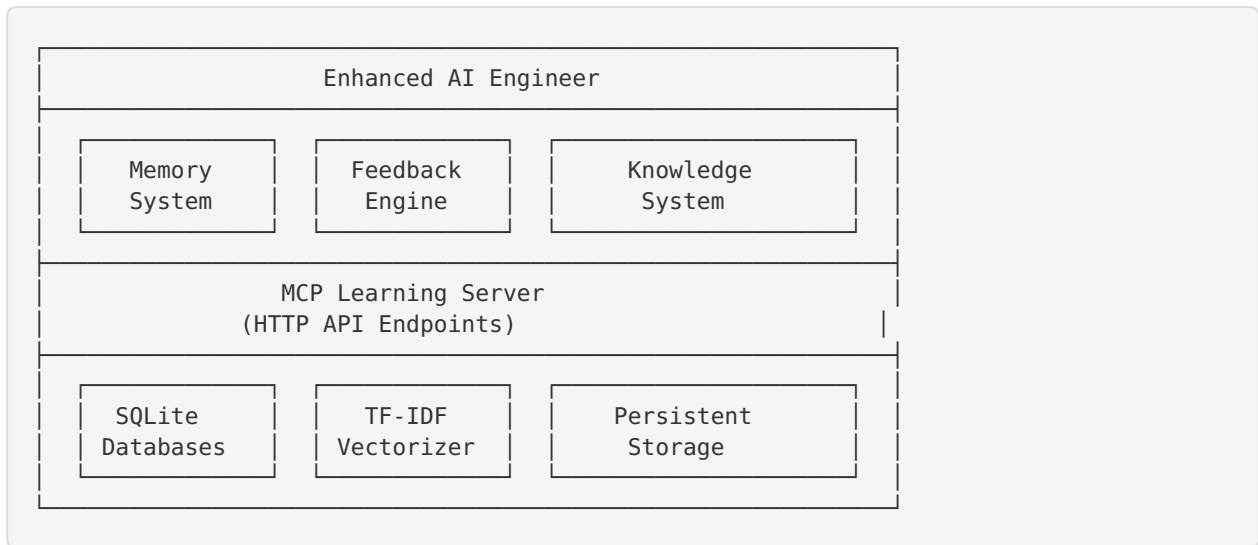
- **RESTful APIs:** Provides HTTP endpoints for all learning capabilities
- **Real-time Updates:** Supports live learning data access
- **Configuration Management:** Allows dynamic system configuration
- **Health Monitoring:** Comprehensive system health and status reporting

6. Enhanced Dashboard (`tk_enhanced_dashboard.py`)

- **Learning Metrics:** Visualizes memory, feedback, and knowledge statistics
- **Real-time Monitoring:** Live updates of learning system status

- **Interactive Controls:** Trigger optimizations and configurations from the UI
- **Knowledge Search:** Built-in knowledge base search functionality

Architecture



Setup and Installation

Prerequisites

- Docker and Docker Compose
- OpenRouter API key (optional, for LLM capabilities)
- GitHub token (optional, for Git operations)

Quick Start

1. Clone and Navigate:

```
bash
git clone <repository-url>
cd quantum-loop-debugger
git checkout self-learning
```

2. Environment Setup:

```
bash
cp .env.example .env
# Edit .env with your API keys
```

3. Launch Enhanced System:

```
bash
docker-compose up -d
```

4. Access Interfaces:

- Enhanced Dashboard: <http://localhost:5000>
- AI Engineer API: <http://localhost:8080>
- MCP Learning Server: <http://localhost:8084>

Manual Installation

If you prefer to run without Docker:

1. Install Dependencies:

```
bash
pip install -r requirements.txt
pip install scikit-learn numpy aiohttp aiohttp-cors requests
```

2. Create Storage Directories:

```
bash
mkdir -p /workspace/{memory, feedback, knowledge, logs}
```

3. Start Services:

```
```bash
Terminal 1: Enhanced AI Engineer
python -m assistant.enhanced_core -mode server -port 8080

Terminal 2: MCP Learning Server
python -m assistant.mcp_learning_server -port 8084

Terminal 3: Enhanced Dashboard
python tk_enhanced_dashboard.py
```
```



Usage Examples

1. Basic Self-Learning Operation

The system automatically learns from every task execution:

```
# Example: The AI Engineer automatically captures this experience
response = await ai_engineer.handle_request({
    'type': 'fix_patch',
    'failure_context': {'error': 'ImportError: No module named requests'},
    'auto_apply': True
})

# The system will:
# 1. Record the experience in memory
# 2. Evaluate the outcome in feedback engine
# 3. Update success patterns
# 4. Adapt future prompts for similar issues
```

2. Interactive Learning Configuration

```
# Configure learning parameters
config_response = await ai_engineer.configure_learning({
    'learning_enabled': True,
    'adaptation_mode': 'automatic',
    'confidence_threshold': 0.8,
    'memory_config': {
        'max_experiences': 5000,
        'similarity_threshold': 0.7
    }
})
```

3. Knowledge Ingestion

```
# Ingest knowledge from various sources
knowledge_response = await ai_engineer.ingest_knowledge(
    source='https://docs.python.org/3/library/requests.html'
)

# Or from local files
knowledge_response = await ai_engineer.ingest_knowledge(
    source='/path/to/documentation.pdf'
)

# Or manual knowledge
knowledge_response = await ai_engineer.ingest_knowledge(
    source='Python Best Practices',
    content='Always use virtual environments...'
)
```

4. Enhanced CLI Interface

```
# Start enhanced CLI
python -m assistant.enhanced_core --mode cli

# Available enhanced commands:
🧠 Enhanced AI Engineer > learning           # Show learning status
🧠 Enhanced AI Engineer > configure {"learning_enabled": true}
🧠 Enhanced AI Engineer > optimize           # Trigger optimization
🧠 Enhanced AI Engineer > ingest https://example.com/docs
```



API Reference

Memory System APIs

Capture Experience

```
POST /memory/capture
Content-Type: application/json

{
  "task type": "fix_patch",
  "context": {"error type": "import error"},
  "action taken": "Added requests to requirements.txt",
  "result": {"success": true},
  "success": true,
  "feedback score": 0.9,
  "tags": ["python", "dependencies"]
}
```

Retrieve Similar Experiences

```
POST /memory/retrieve
Content-Type: application/json

{
  "query_context": {"error type": "import error"},
  "task type": "fix_patch",
  "limit": 5,
  "success only": true
}
```

Get Adaptive Prompt

```
POST /memory/adaptive_prompt
Content-Type: application/json

{
  "task type": "fix_patch",
  "context": {"error": "ImportError"},
  "base prompt": "Fix the following error..."
}
```

Feedback Engine APIs

Record Feedback

```
POST /feedback/record
Content-Type: application/json

{
  "task id": "task 123",
  "task type": "fix_patch",
  "action": "Applied dependency fix",
  "result": {"success": true},
  "execution time": 15.5,
  "user feedback": {"satisfaction": 0.9}
}
```

Get Performance Summary

```
GET /feedback/summary?task_type=fix_patch
```

Get Learning Insights

```
GET /feedback/insights
```

Knowledge System APIs

Ingest Knowledge

```
POST /knowledge/ingest
Content-Type: application/json

{
  "source": "https://example.com/docs",
  "content": "Optional pre-extracted content"
}
```

Search Knowledge

```
POST /knowledge/search
Content-Type: application/json

{
  "query": "python import errors",
  "limit": 10,
  "min_importance": 0.5,
  "tags": ["python", "debugging"]
}
```

Identify Knowledge Gaps

```
POST /knowledge/gaps
Content-Type: application/json

{
  "context": {
    "failures": [
      {
        "error": "ModuleNotFoundError",
        "task type": "fix patch"
      }
    ]
  }
}
```



Monitoring and Analytics

Dashboard Metrics

The enhanced dashboard provides comprehensive monitoring:

1. **System Status:** Overall health and uptime
2. **OpenRouter Status:** Model rotation and usage
3. **Learning Metrics:** Experiences, success rates, adaptations
4. **Memory System:** Experience count, success patterns, storage usage
5. **Feedback Engine:** Performance trends, optimization opportunities
6. **Knowledge System:** Knowledge items, gaps, improvement proposals

Performance Tracking

Key metrics automatically tracked:

- **Success Rate:** Percentage of successful task completions
- **Learning Velocity:** Rate of improvement over time
- **Adaptation Effectiveness:** Impact of learning on performance
- **Knowledge Coverage:** Areas with sufficient vs. insufficient knowledge
- **Memory Utilization:** Storage and retrieval efficiency

Troubleshooting

Common Issues

1. Memory System Not Learning:

```
```bash
Check if learning is enabled
curl -X GET http://localhost:8084/status

Enable learning if disabled
curl -X POST http://localhost:8080/api/request \
-H "Content-Type: application/json" \
-d '{"type": "configure_learning", "config": {"learning_enabled": true}}'
```
```

1. Knowledge Ingestion Failing:

```
```bash
Check knowledge system status
curl -X GET http://localhost:8084/knowledge/stats

Test with simple text ingestion
curl -X POST http://localhost:8084/knowledge/ingest \
-H "Content-Type: application/json" \
-d '{"source": "test", "content": "This is a test knowledge item"}'
```
```

1. Dashboard Not Showing Learning Data:

- Verify MCP Learning Server is running on port 8084
- Check Docker container logs: `docker logs quantum-mcp-learning`
- Ensure persistent volumes are properly mounted

Debugging

Enable debug logging:

```
# Set environment variable
export PYTHONPATH=/workspace
export LOG_LEVEL=DEBUG

# Or modify logging in code
import logging
logging.basicConfig(level=logging.DEBUG)
```

Advanced Configuration

Memory System Tuning

```
# Adjust memory system parameters
memory_config = {
    'max_experiences': 10000,      # Maximum experiences to store
    'similarity_threshold': 0.7,   # Minimum similarity for retrieval
    'adaptation_threshold': 0.8,   # Minimum score for prompt adaptation
    'importance_decay_days': 90    # Days before importance decays
}
```

Feedback Engine Optimization

```
# Configure feedback evaluation
feedback_config = {
    'evaluation_window': 100,      # Recent events to consider
    'optimization_threshold': 0.7, # Success rate trigger for optimization
    'learning_rate': 0.1,         # Rate of adaptation
    'feedback_weights': {         # Importance of different factors
        'success': 0.4,
        'execution_time': 0.2,
        'user_feedback': 0.3,
        'error_severity': 0.1
    }
}
```

Knowledge System Settings

```
# Knowledge system configuration
knowledge_config = {
    'max_knowledge_items': 50000,  # Maximum knowledge items
    'similarity_threshold': 0.6,    # Knowledge similarity threshold
    'supported_formats': ['.txt', '.md', '.pdf', '.json'],
    'importance_decay_days': 90
}
```

Future Enhancements

The self-learning framework is designed for extensibility. Planned enhancements include:

1. **Advanced ML Integration:** Reinforcement learning for decision optimization
2. **Multi-Agent Learning:** Collaborative learning across multiple AI Engineer instances
3. **Real-time Adaptation:** Immediate learning from user feedback
4. **Knowledge Graph Visualization:** Interactive knowledge relationship mapping
5. **Predictive Failure Detection:** Proactive issue identification
6. **A/B Testing Framework:** Systematic approach comparison

Contributing

To contribute to the self-learning capabilities:

1. Fork the repository

2. Create a feature branch: `git checkout -b feature/learning-enhancement`
3. Implement your enhancement
4. Add tests and documentation
5. Submit a pull request

Development Guidelines

- Follow existing code patterns and documentation standards
- Ensure backward compatibility with the original AI Engineer
- Add comprehensive logging for debugging
- Include unit tests for new functionality
- Update this guide with new features



License

This enhanced self-learning framework maintains the same license as the original Quantum Loop Debugger project.

 **The Enhanced AI Engineer with Self-Learning Capabilities - Continuously Improving, Zero-Cost Operation**